

Hibernate

16 July 2024 11:14

- Introduction
- Why hibernate
- Advantages of hibernate
- Architecture
- POJO class
- Mapping file(XML)
- Configuration file
- Hibernate examples(CRUD operations)
- Hibernate Query language (HQL)
- Hibernate criteria query language
- Inheritance mapping

Introduction

16 July 2024 12:02

- Hibernate is a java framework is developed by gavin king.
- Main objective of JPA and hibernate is to use to execute queries automate.....
- It is a framework which simplifies the development of java application to interact with a database.
- It is a open source, lightweight, and works based on ORM tool.
 - o ORM tool:- object relational mapping
 - ORM is a technique for converting data between java objects and relational databases.
 - ORM implements responsibility of mapping the object-relational model.

JAVA APPLICATION	----ORM TOOL----	DATABASE
------------------	------------------	----------

- Some of the popular ORM tools are
 - ☐ Hibernate-co1
 - ☐ iBitis
 - ☐ MyBTS
 - ☐ TopLink

Why hibernate

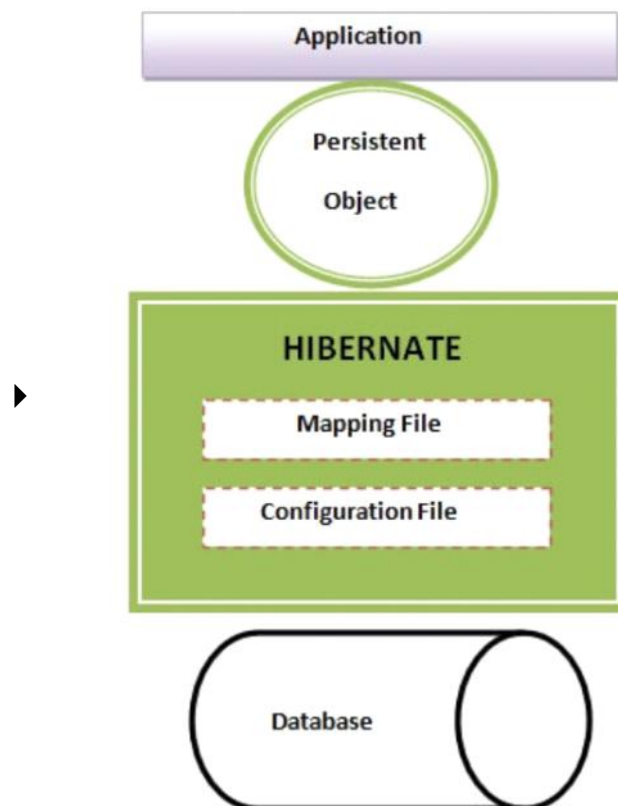
16 July 2024 12:20

- Simplifies database interactions.
- Cross database portability

ADVANTAGES of HIBERNATE:

- Open source and light weight
- Fast performance
- Database independent query
- Automate table creation
- Throws exceptions by default

ARCHITECTURE



- It consists of three layers..
 - i. Application layer (java application)
 - ii. Hibernate layer (mapping , configuration)
 - iii. Database layer(Oracle, MySQL)
 - Hibernate architecture consists of pre-defined java objects or interfaces such as
 - i. SessionFactory
 - ii. ConfigurationFactory
 - iii. TransactionFactory
 - POJO class:- POJO standards for Plain Old Java Object
 - POJO is nothing but a java bean.
 - Hibernate allows only POJO classes.
 - POJO class consists of setters and getters.
 - Example
- ```
Class student{
 Private int id;
```

```

 Private string name;
 Private double cgpa;
 //all t
}

```

- Configuration file:- the purpose of configuration of file is to define the properties of database
  - Configuration file can be define in two ways:
    - It will be either xml or annotation
  - The configuration file has to be denoted by hibernate.cfg.xml
  - Configuration file is loaded into hibernate application during the runtime of a project
  - The configuration file must contain the following information
    - i. Connection properties
    - ii. Hibernate properties
    - iii. Mapping file resources
  - Note :- number of configuration files is **equals** to number of databases

Syntax:-

Hibernate.cfg.xml

```

<hibernate-configuration>
 <session-factory>
 <!-- connection properties-->
 <property name="connection.driver_class">Load Drivers</property>
 <property name="connection.url">Connection Establishment</property>
 <property name="connection.username">User name</property>
 <property name="connection.password">Password</property>

 <!--hibernate properties--->
 <property name="show_sql"> true/false</property>
 <property name="dialect">database name</property>
 <property name="hb2mddl.auto"> create </property>

 <!--mapping-file-->
 <mapping resource="filename(mapping)"/>
 <mapping resource="filename(mapping)"/>
 <mapping resource="filename(mapping)"/>

 <!--no of mapping files is equal to number of tables-->

 </session-factory>
</hibernate-configuration>

```

# Example-oracle

22 July 2024 11:58

Example:- Oracle

```
<hibernate-configuration>
 <session-factory>
 <!-- connection properties-->
 <property name="connection.driver_class">
 Oracle.jdbc.driver.oracleDriver</property>
 <property name="connection.url">jdbc:oracle:thin:
 @localhost:1521:xe</property>
 <property name="connection.username">system</property>
 <property name="connection.password">admin</property>

 <!--hibernate properties--->
 <property name="show_sql"> true</property>
 <property name="dialect">org.hibernate.oracle</property>
 <property name="hb2mddl.auto"> create </property>

 <!--mapping-file-->
 <mapping resource="employee1.hbm.xml"/>
 <mapping resource="employee1.hbm.xml"/>

 <!--no of mapping files is equal to number of tables-->

 </session-factory>
</hibernate-configuration>
```

# Ex-MySQL

22 July 2024 12:03

Example:-

```
<hibernate-configuration>
 <session-factory>
 <!-- connection properties-->
 <property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
 <property name="connection.url">jdbc:mysql://localhost:3306/co1</property>
 <property name="connection.username">root</property>
 <property name="connection.password">admin</property>

 <!--hibernate properties--->
 <property name="show_sql"> true</property>
 <property name="dialect">org.hibernate.mysql</property>
 <property name="hb2mddl.auto"> create </property>

 <!--mapping-file-->
 <mapping resource="employee1.hbm.xml"/>
 <mapping resource="employee2.hbm.xml"/>

 <!--no of mapping files is equal to number of tables-->

 </session-factory>
</hibernate-configuration>
```



# Mapping file

22 July 2024 12:15

- It is a part of hibernate application
- Mapping file can be implemented either in xml(recommended) or annotations.
- Every ORM needs... it is a mechanism of placing object properties(java object) to the specific column of a table.
- This mapping file contains how a mapping can be done from a pojo class to database name and from class properties to column names.

POJO class	Table name
Prop-1	Col-1
Prop-2	Col-2

- While creating a mapping file we can create one or multiple number of mapping files based on application requirements.
- Note :-
  - o My java object will interact with table
  - o Every object will have the following properties
    - i. Identity(Object Name)
    - ii. State (Object Value)
    - iii. Behaviour (Object Method)
  - o Syntax:- filename.hbm.xml

```
<hibernate-mapping>
<class name="POJO class name" table="table-name">
 <id name="class-property" column="col name in table "></id>
 <property name="class-property" column="col name in table "></property>
</class>
</hibernate-mapping>
```
  - o Example: employee.hbm.xml

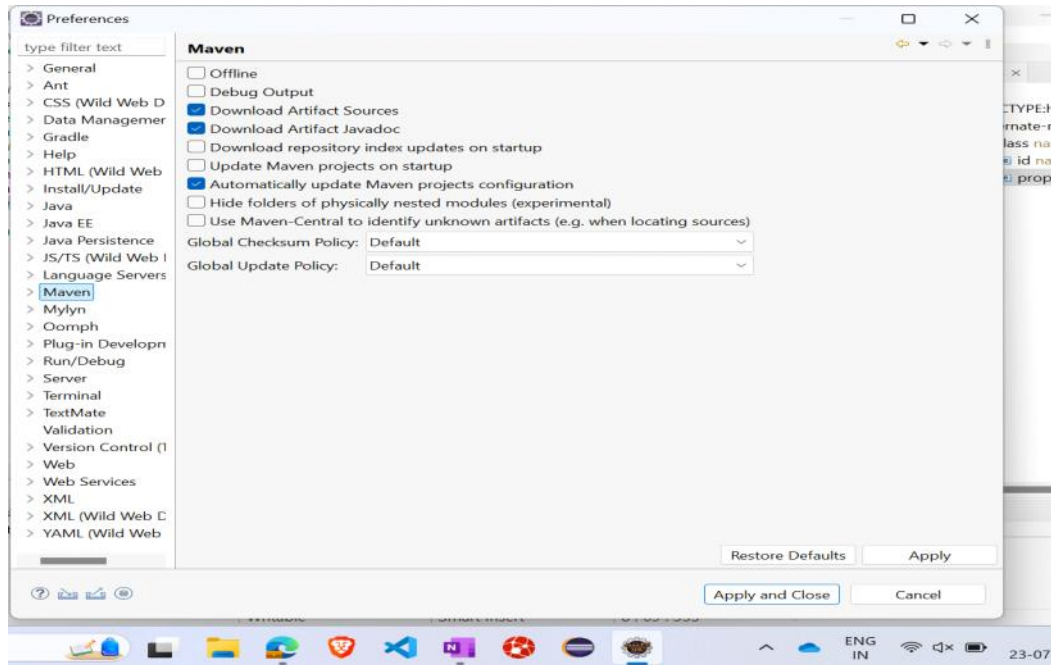
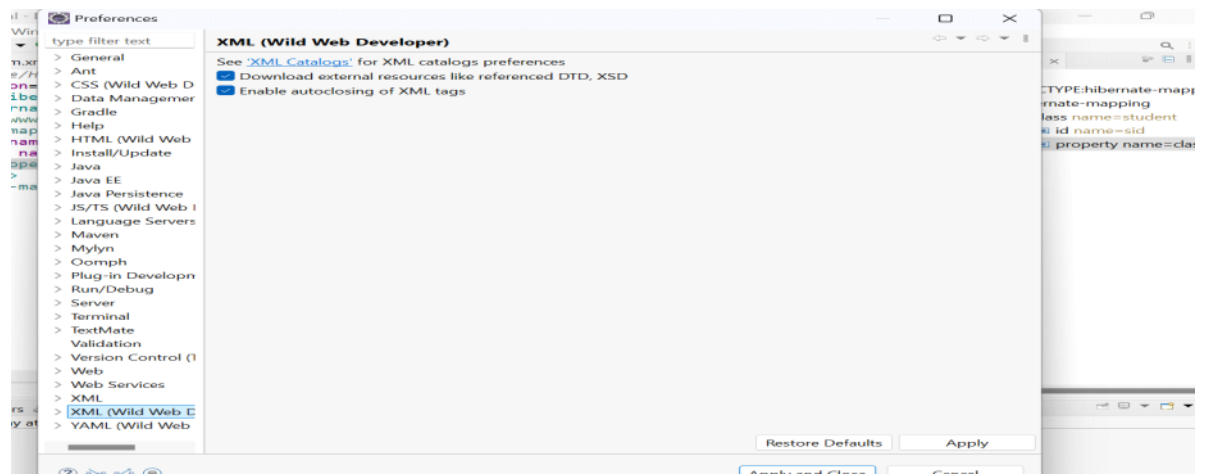
```
<hibernate-mapping>
<class name="student" table="student">
 <id name="sid" column="sid "></id>
 <property name="class-property" column="col name in table "></property>
</class>
</hibernate-mapping>
```
  - o Table is created by user with table name as std and column names as tid and tname
  - o Note:- significance of hibernate is (Table must be created automatically)
    - Syntax:-

```
<hibernate-mapping>
<class name="POJO class name">
<id name="class-property"/>
<property name="class-property"/>
</class>
</hibernate-mapping>
```

Employee1.hbm.xml

```
<hibernate-mapping>
<class name="Employee">
<id name="id"/>
<property name="ename"/>
</class>
</hibernate-mapping>
```
    - Table will be created automatically by hibernate framework with table as employee and column as eid and ename.
    - Preferences--->wild web developer-->turn on





# CRUD(Hibernate)

22 July 2024 12:41

- Every hibernate application must have the following four files
  - a. POJO class(.java)
  - b. Configuration file(.xml)
  - c. Mapping file(filename.hbm.xml)
  - d. Logic file (.java) (main method) (execute)
- Skeleton of hibernate application
- Step-1: Create a maven project.
  - Archetype:: maven-archetype-quickstart
- Step-2: pom.xml
  - 1.7-->1.8
  - Add dependencies
    - Hibernate core
    - Hibernate entity manager
    - MySQL/Oracle
- Step-3: update maven project
- Step-4: you to need to create a new folder as 'resources'(configuration and mapping file)
  - ->src->main->right click ----->new----->folder(resources)
- Step-5: Implement Hibernate Concepts
  - POJO class(src/main/java)---entity.java
  - Configuration file(src/main/resources)--- hibernate.cfg.xml
  - Mapping file(src/main/resources)--- hibernate.hbm.xml
  - Logic file (src/main/java)---(filename.java)
- Step-6: Run Logic file
  - Right Click-->run as --->java application
  
- Example-1:
  - Hibernate Example-(Insert)

```
To insert multiple values at a time;
Configuration cfg = new Configuration();
cfg.configure("hibernate.cfg.xml");

SessionFactory sf = cfg.buildSessionFactory();

int j = 1230;

for (int i = 120; i < 131; i++) {
 Session s = sf.openSession();
 Employee emp = new Employee();
 Transaction t = s.beginTransaction();
 emp.setEid(i);
 j = j + i;
 emp.setEname("murthy");
 emp.setEsal(j);
}
```

```
 s.persist(emp);
 t.commit();
 s.close();
 }

 sf.close();
```

# HQL

29 July 2024 11:29

- HQL stands for hibernate query language
- HQL is database independent query language
- HQL is same as SQL, but the only difference is SQL depends on table, where as HQL depends on POJO class
- To work with HQL we need to use Query interface

## Query Interface

- It is an object oriented representation of hibernate query
- The object of a query interface can be obtained by calling create query method to a session object.
- The methods of query interface are
  - a. Execute Update()
  - b. List()
  - c. setFirstResult()
  - d. setMaxResult()
  - e. setParameter()
- Example-1:- HQL example (To retrieve all the records in for each)  
POJO class -->  
Logic file
- Example-1:- HQL example (To retrieve all the records in ITERATOR)
- Example-3: HQL Example to Retrieve specific range (starting record to how many number of records)  
POJO CLASS  
Configuration file  
Mapping files  
Logic file(HQLRange)

Query q=---->>

q.setFirstResult(5);

q.setMaxResult(15); //starting from 5 there it will execute upto 15

Example-4:- HQL example

(To update record)

HQL-->> update

Query --> update Employee set ename = :n where eid=:l

q.setParameter(n,"XYZ");

q.setParameter(i,111);

Example-5: HQL delete

Query --> delete from employee where id="111";

Example-6:-- HQL example (to insert a record)

Query==>> insert into Employee(eid,ename,esal) values

(111,"QWERTY",7000)

Example-7:= HQL Example (To retrieve all records with partial number of columns)

Eid,ename,esal-->Employee

Eid,ename-->HQL

Example-8 ==> HQL example( to retrieve all records with single column)

- Hibernate Criteria Query Language
- Using hcql I can able to reterive the records with all number of columns every time.
- In order to implement hcql in hibernate application, I need to use criteria interface

## **Criteria Interface**

- Criteria interface object can be obtain by calling .createCriteria method to a session object  
Criteria cr = s.createCriteria();
- The methods of criteria interface are
  - o Add
  - o addOrder
  - o List
  - o setFirstResult
  - o setMaxResults

## **Restriction class**

- This is the class which consists of many methods in order add restrictions/conditions/criteria on HQL query
- The popular methods of restriction class are
  - o Lt
  - o le
  - o Gt
  - o Ge()
  - o Eq()
  - o Ne()

## **Order class**

- In order to reterive data from db table either in asc or dsc order
- The methods are
  - o Asc()
  - o Dsc()

1. To reterive records using for Each

# Inheritance mapping

05 August 2024 11:14

## Inheritance:-

- Acquiring the properties from the **base** class to **derived** class refers to inheritance

Note: Inheritances are five types with respect to OOPS

1. Single inheritance: base<--->derived
2. Multi inheritance : Base-1,Base-2,Base-3--->> Derived-1 .. In java multi inheritance can be achieved by interface
3. Multi level inheritance: Base-1--->>Intermediate-1--->>Derived 1
4. Hierarchal inheritance: Base-1--->Derived-1,derived-2,derived-3
5. Hybrid Inheritance : combination of two inheritances i.e, hierarchal inheritance and multiple inheritance.  
Base-1--->>intermediate-1,intermediate-2,intermediate-3----->>>>derived-1

## Mapping:-

- It refers to relationship between different tables in database.
- Different types of mappings are
  1. IS-A(Inheritance)
  2. HAS-A(Association)

Inheritance mapping can be implemented in 3 ways in hibernate application

- Table per class
- Table per sub-class
- Table per concrete-class

## Scenario:-

- Payment (base)---->>Card(derived-1),cheque(derived-2)

-

Person(base)---->Student(Derived-1),Employee(Derived-2)`

1. Table per class:-Table will be created for {Payment<===>person}
2. Table per sub class: Table will be created for { card,cheque<===>Student,Employee}
3. Table per concrete class: Table will be created for { Payment,card,cheque<===>person,student,employee}



POJO class:-

Payment.java

```
Class Payment{
 Int pid;
 Double pamount;

 -->getters and setters
}
```

Card.java

```
Class Card{
 String cardType;
 //generate getters and setters
}
```

Cheque.java

```
Class cheque{
 String chequeType;
 //generate getters and setters
}
```

Configuration file:-

- Hibernate.cfg.xml
- Connection properties
- Hibernate properties
- Mapping properties

Mapping file:- payment.hbm.xml

```
<hibernate-mapping>
<class name="com.klu.JFSDS25">

<id name="pid"/>
<property name="pamount"/>
<subclass name="com.klu.JFSDS25_IMTC.Card" discriminator="c">
<property name="cardType"/>

</subclass>
<subclass name="com.klu.JFSDS25_IMTC.Cheque" discriminator="cq">
<property name="Cheque"/>

</subclass>

</class>
</hibernate-mapping>
```

```

Logic file(Table per class):-
Class TaberPerClass{
Main(){
Configuration cfg = new Configuration;
Cfg.configure("hibernate.cfg.xml")
SessionFactory sf = cfg.buildSessionFactory();
Session s= sf.openSession();
Transaction t= s.beginTransaction();
Card c = new Card();
Cheque cq = new Cheque();
c.setPid(101);
c.setPamount(145000);
c.setCardType("Credit");

cq.setPid(102);
cq.setPamount(145000);
cq.setChequeType("RTS");

}}

```

```

mysql> use hibcrud;
Database changed
mysql> show tables;
+-----+
| Tables_in_hibcrud |
+-----+
| employe |
| employee |
| payment |
+-----+
3 rows in set (0.01 sec)

mysql> select * from payment;
+----+-----+-----+-----+-----+
| pid | payment_type | pamount | card_type | cheque_type |
+----+-----+-----+-----+-----+
| 101 | c | 145000 | Credit | NULL |
| 102 | cq | 145000 | NULL | RTS |
+----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

### Example-2:- table per sub class

Configuration file--prev

Pojo class

Payment -->previous

Card --> same as previous

Cheque-->same as previous

```

<hibernate-mapping>
<class name="com.klu.JFSDS25">

<id name="pid"/>
<property name="pamount"/>
<JoinedClass name="com.klu.JFSDS25_IMTC.Card" discriminator="c">
<property name="cardType"/>

</subclass>
<JoinedClass name="com.klu.JFSDS25_IMTC.Cheque" discriminator="cq">
<property name="Cheque"/>

</subclass>

</class>
</hibernate-mapping>

```

Logic file:- same as previous

```

+-----+
| Tables_in_hibcrud |
+-----+
| employee |
| employee |
| payment |
+-----+
3 rows in set (0.01 sec)

mysql> select * from payment;
+-----+-----+-----+-----+-----+
| pid | payment_type | pamount | card_type | cheque_type |
+-----+-----+-----+-----+-----+
| 101 | c | 145000 | Credit | NULL |
| 102 | cq | 145000 | NULL | RTS |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> select * from payment;
+-----+-----+-----+-----+-----+
| pid | payment_type | pamount | card_type | cheque_type |
+-----+-----+-----+-----+-----+
| 101 | c | 145000 | Credit | NULL |
| 102 | cq | 145000 | NULL | RTS |
| 104 | c | 15000 | debit | NULL |
| 105 | cq | 13530 | NULL | RTS |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>

```

**Example -3: table per concrete class**

Configuration file--prev

Pojo class

Payment -->previous

Card --> same as previous

Cheque-->same as previous

<hibernate-mapping>

<class name="com.klu.JFSDS25">

<id name="pid"/>

<property name="pamount"/>

<UnionClass name="com.klu.JFSDS25\_IMTC.Card" discriminator="c">

<property name="cardType"/>

</subclass>

<unionClass name="com.klu.JFSDS25\_IMTC.Cheque" discriminator="cq">

<property name="Cheque"/>

</subclass>

</class>

</hibernate-mapping>

Logic file:- same as previous

# Table per class

09 August 2024 19:26

## Pojo class

Payment.java

```
package inheritanceMapping.com.payment;
```

```
public class Payment {
 int pid;
 double pamount;
 public int getPid() {
 return pid;
 }
 public void setPid(int pid) {
 this.pid = pid;
 }
 public double getPamount() {
 return pamount;
 }
 public void setPamount(double pamount) {
 this.pamount = pamount;
 }
}
```

Card.java

```
package inheritanceMapping.com.payment;
```

```
public class Card extends Payment{
 String CardType;

 public String getCardType() {
 return CardType;
 }

 public void setCardType(String cardType) {
 this.CardType = cardType;
 }
}
```

Cheque.java

```
package inheritanceMapping.com.payment;
```

```
public class Cheque extends Payment{
 String ChequeType;

 public String getChequeType() {
 return ChequeType;
 }

 public void setChequeType(String ChequeType) {
 this.ChequeType = ChequeType;
 }
}
```

```
}
```

hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
<session-factory>
<!-- Connection properties -->
<property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
<property name="connection.url">jdbc:mysql://localhost:3306/Hibcrud</property>
<property name="connection.username">root</property>
<property name="connection.password">admin</property>
<!-- Hibernate properties -->
<property name="show_sql">>true</property>
<property name="dialect">org.hibernate.dialect.MySQL5Dialect</property>
<property name="hibernate.hbm2ddl.auto">update</property>
<!-- Mapping files -->
<!-- <mapping resource="payment.hbm.xml"/> -->
<!-- <mapping resource="subclass.hbm.xml"/> -->
<mapping resource="concrete.hbm.xml"/>
</session-factory>
</hibernate-configuration>
```

Payment.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
<class name="inheritanceMapping.com.payment.Payment" table="payment">
<id name="pid" column="pid"/>
<!-- Discriminator should appear here -->
<discriminator column="payment_type" type="string"/>
<property name="pamount" column="pamount"/>

<subclass name="inheritanceMapping.com.payment.Card" discriminator-value="c">
<property name="cardType" column="card_type"/>
</subclass>

<subclass name="inheritanceMapping.com.payment.Cheque" discriminator-value="cq">
<property name="chequeType" column="cheque_type"/>
</subclass>
</class>
</hibernate-mapping>
```

TablePerClass.java-- logic file

```
package inheritanceMapping.com.payment;
```

```
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
```

```

public class TablePerClass {

 public static void main(String[] args) {
 Configuration cfg = new Configuration();
 cfg.configure("hibernate.cfg.xml");
 SessionFactory sf = cfg.buildSessionFactory();
 Session s= sf.openSession();
 Transaction t= s.beginTransaction();

 Card c = new Card();
 Cheque cq = new Cheque();
 c.setPid(101);
 c.setPamount(145000);
 c.setCardType("Credit");

 cq.setPid(102);
 cq.setPamount(145000);
 cq.setChequeType("RTS");

 s.persist(c);
 s.persist(cq);
 t.commit();

 }

}

```

```

mysql> use hibcrud;
Database changed
mysql> show tables;
+-----+
| Tables_in_hibcrud |
+-----+
| employee |
| employee |
| payment |
+-----+
3 rows in set (0.01 sec)

mysql> select * from payment;
+-----+-----+-----+-----+-----+
| pid | payment_type | pamount | card_type | cheque_type |
+-----+-----+-----+-----+-----+
| 101 | c | 145000 | Credit | NULL |
| 102 | cq | 145000 | NULL | RTS |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

# Table per sub class

09 August 2024

19:32

## Pojo class

Payment.java

```
package inheritanceMapping.com.payment;
```

```
public class Payment {
 int pid;
 double pamount;
 public int getPid() {
 return pid;
 }
 public void setPid(int pid) {
 this.pid = pid;
 }
 public double getPamount() {
 return pamount;
 }
 public void setPamount(double pamount) {
 this.pamount = pamount;
 }
}
```

}

Card.java

```
package inheritanceMapping.com.payment;
```

```
public class Card extends Payment{
 String CardType;

 public String getCardType() {
 return CardType;
 }

 public void setCardType(String cardType) {
 this.CardType = cardType;
 }
}
```

}

Cheque.java

```
package inheritanceMapping.com.payment;
```

```
public class Cheque extends Payment{
 String ChequeType;

 public String getChequeType() {
 return ChequeType;
 }

 public void setChequeType(String ChequeType) {
 this.ChequeType = ChequeType;
 }
}
```



```
}
```

hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
<session-factory>
<!-- Connection properties -->
<property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
<property name="connection.url">jdbc:mysql://localhost:3306/Hibcrud</property>
<property name="connection.username">root</property>
<property name="connection.password">admin</property>
<!-- Hibernate properties -->
<property name="show_sql">>true</property>
<property name="dialect">org.hibernate.dialect.MySQL5Dialect</property>
<property name="hibernate.hbm2ddl.auto">update</property>
<!-- Mapping files -->
<!-- <mapping resource="payment.hbm.xml"/> -->
<!-- <mapping resource="subclass.hbm.xml"/> -->
<mapping resource="concrete.hbm.xml"/>
</session-factory>
</hibernate-configuration>
```

Sub-class.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
<class name="inheritanceMapping.com.payment.Payment" table="payment">
<id name="pid" column="pid">
<generator/>
</id>
<property name="pamount" column="pamount"/>

<joined-subclass name="inheritanceMapping.com.payment.Card" table="card">
<key column="pid"/>
<property name="cardType" column="card_type"/>
</joined-subclass>

<joined-subclass name="inheritanceMapping.com.payment.Cheque" table="cheque">
<key column="pid"/>
<property name="chequeType" column="cheque_type"/>
</joined-subclass>
</class>
</hibernate-mapping>
```

TablePerClass.java-- logic file

```
package inheritanceMapping.com.payment;
```

```
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
```

```

import org.hibernate.cfg.Configuration;

public class TablePerClass {

 public static void main(String[] args) {
 Configuration cfg = new Configuration();
 cfg.configure("hibernate.cfg.xml");
 SessionFactory sf = cfg.buildSessionFactory();
 Session s = sf.openSession();
 Transaction t = s.beginTransaction();

 Card c = new Card();
 Cheque cq = new Cheque();
 c.setPid(101);
 c.setPamount(145000);
 c.setCardType("Credit");

 cq.setPid(102);
 cq.setPamount(145000);
 cq.setChequeType("RTS");

 s.persist(c);
 s.persist(cq);
 t.commit();
 }
}

```

```

+-----+
| Tables_in_hibcrud |
+-----+
| employe |
| employee |
| payment |
+-----+

```

3 rows in set (0.01 sec)

mysql> select \* from payment;

```

+-----+-----+-----+-----+-----+
| pid | payment_type | pamount | card_type | cheque_type |
+-----+-----+-----+-----+-----+
| 101 | c | 145000 | Credit | NULL |
| 102 | cq | 145000 | NULL | RTS |
+-----+-----+-----+-----+-----+

```

2 rows in set (0.00 sec)

mysql> select \* from payment;

```

+-----+-----+-----+-----+-----+
| pid | payment_type | pamount | card_type | cheque_type |
+-----+-----+-----+-----+-----+
| 101 | c | 145000 | Credit | NULL |
| 102 | cq | 145000 | NULL | RTS |
| 104 | c | 15000 | debit | NULL |
| 105 | cq | 13530 | NULL | RTS |
+-----+-----+-----+-----+-----+

```

4 rows in set (0.00 sec)

mysql> |

# Concrete class

09 August 2024 19:34

## Pojo class

Payment.java

```
package inheritanceMapping.com.payment;
```

```
public class Payment {
 int pid;
 double pamount;
 public int getPid() {
 return pid;
 }
 public void setPid(int pid) {
 this.pid = pid;
 }
 public double getPamount() {
 return pamount;
 }
 public void setPamount(double pamount) {
 this.pamount = pamount;
 }
}
```

}

Card.java

```
package inheritanceMapping.com.payment;
```

```
public class Card extends Payment{
 String CardType;

 public String getCardType() {
 return CardType;
 }

 public void setCardType(String cardType) {
 this.CardType = cardType;
 }
}
```

}

Cheque.java

```
package inheritanceMapping.com.payment;
```

```
public class Cheque extends Payment{
 String ChequeType;

 public String getChequeType() {
 return ChequeType;
 }

 public void setChequeType(String ChequeType) {
 this.ChequeType = ChequeType;
 }
}
```

```
}
```

#### hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
" http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd>

<hibernate-configuration>
<session-factory>
<!-- Connection properties -->
<property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
<property name="connection.url">jdbc:mysql://localhost:3306/Hibcrud</property>
<property name="connection.username">root</property>
<property name="connection.password">admin</property>
<!-- Hibernate properties -->
<property name="show_sql">true</property>
<property name="dialect">org.hibernate.dialect.MySQL5Dialect</property>
<property name="hibernate.hbm2ddl.auto">update</property>
<!-- Mapping files -->
<!-- <mapping resource="payment.hbm.xml"/> -->
<!-- <mapping resource="subclass.hbm.xml"/> -->
<mapping resource="concrete.hbm.xml"/>
</session-factory>
</hibernate-configuration>
```

#### concrete.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
" http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd>

<hibernate-mapping>
<class name="inheritanceMapping.com.payment.Payment" table="payment">
<id name="pid" column="pid"/>
<property name="pamount" column="pamount"/>

<!-- Union-subclass for Card -->
<union-subclass name="inheritanceMapping.com.payment.Card" table="card">
<property name="cardType" column="card_type"/>
</union-subclass>

<!-- Union-subclass for Cheque -->
<union-subclass name="inheritanceMapping.com.payment.Cheque" table="cheque">
<property name="ChequeType" column="cheque_type"/>
</union-subclass>
</class>
</hibernate-mapping>
```

#### TablePerClass.java-- logic file

```
package inheritanceMapping.com.payment;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
```

```

import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class TablePerClass {

 public static void main(String[] args) {
 Configuration cfg = new Configuration();
 cfg.configure("hibernate.cfg.xml");
 SessionFactory sf = cfg.buildSessionFactory();
 Session s= sf.openSession();
 Transaction t= s.beginTransaction();

 Card c = new Card();
 Cheque cq = new Cheque();
 c.setPid(101);
 c.setPamount(145000);
 c.setCardType("Credit");

 cq.setPid(102);
 cq.setPamount(145000);
 cq.setChequeType("RTS");

 s.persist(c);
 s.persist(cq);
 t.commit();

 }

}

```

104	c	15000	debit	NULL
105	cq	13530	NULL	RTS
109	default_value	15000	NULL	NULL
115	default_value	13530	NULL	NULL
139	default_value	15000	NULL	NULL
122	default_value	13530	NULL	NULL

8 rows in set (0.00 sec)

mysql> select \* from card;

pid	card_type	pamount
109	debit	NULL
139	debit	NULL
1390	yono	15400

3 rows in set (0.00 sec)

mysql> select \* from cheque;

pid	cheque_type	pamount
115	RTS	NULL
122	RTS	NULL
142	BTS	13130

3 rows in set (0.00 sec)

# Spring

13 August 2024

11:27

- ☺ Introduction
- ☺ Why spring
- ☺ Advantages of spring
- ☺ Spring architecture
- ☺ Dependency injection(DI)
- ☺ Inversion of control(IOC)
- ☺ Primitive and non-primitive data types
- ☺ Spring DAO using JDBC
- ☺ Spring DAO using Hibernate
- ☺ Spring MVC



# Introduction

13 August 2024 11:34

- ✧ Spring is a java framework which was developed by ROD JOHNSON in 2003
- ✧ This framework is used to build a java application at enterprise level(We can build console applications and web applications)
- ✧ It is a light weight framework
- ✧ Applications design using spring framework executes faster.
- ✧ The core principle of spring framework is to simplify the java development process and to promote good design practices.
- ✧ Spring framework comprises of several modules such as IOC,DAO,ORM,WEB-MVC etc

## WHY SPRING

- Using hibernate we can build only console application whereas using spring framework, we can able to build applications from console to web.
- Hibernate is a ORM framework, primarily focused on data persistence. Whereas spring is a comprehensive framework that provide array of features.
- Spring can be used various ORM frameworks including hibernate.(Spring is called as master framework or mother framework)

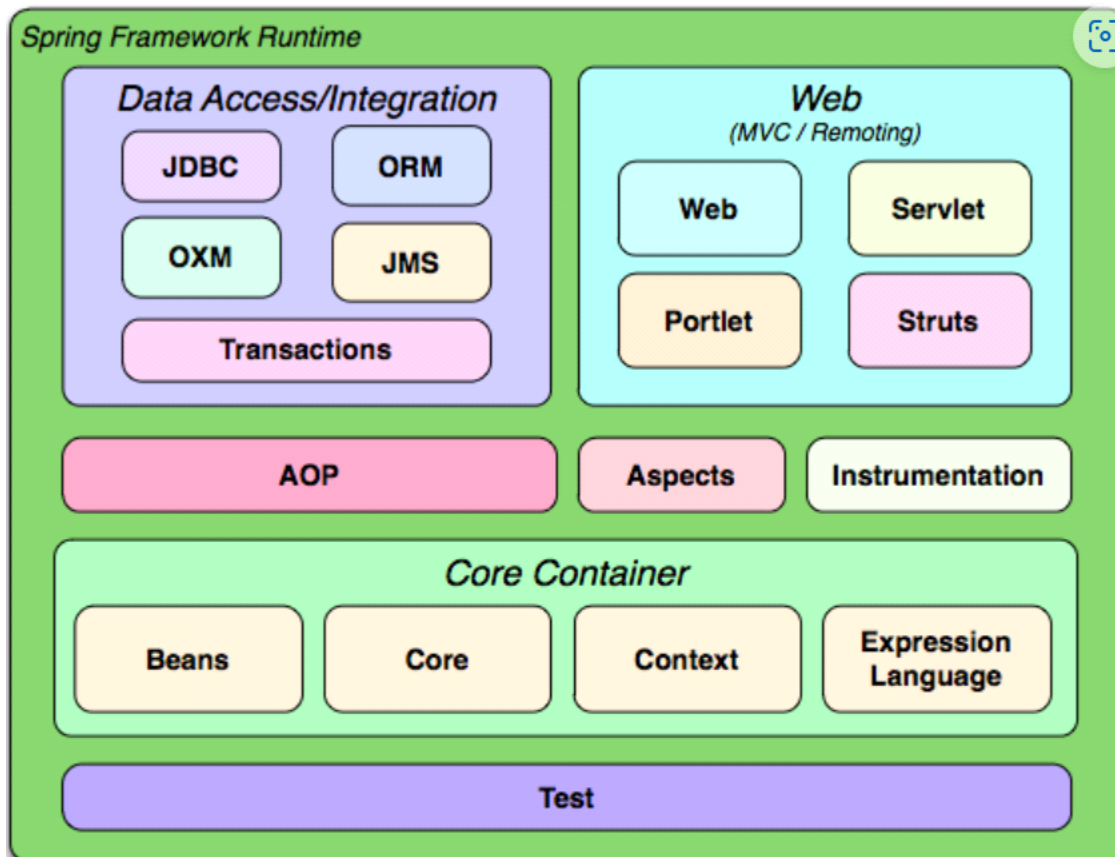
## Advantages of spring

- It is easy to test and maintain
- Loosely coupled(one component independent to other)
- It provides pre-defined templates

# Architecture

13 August 2024 12:42

- Spring architecture consists of several modules such as IOC, DAO, ORM, Web MVC
- Java is also known as aspect oriented programming(AOP)
- All these modules are grouped into Test, spring core container, DAO, WEB MVC etc.
- DOCUMENTATION. [1. Introduction to Spring Framework](#)
- POJO:plain old java objects



# LooselyCoupling

17 August 2024 14:22

Note-1:-

Program:

2 classes (one class depends on other class)--tightly coupled

2 classes(one class not depends on other class)-- Loosely Coupled

Tightly coupled

// Tightly Coupled Example

```

```
class Engine {
    public void start() {
        System.out.println("Engine is starting...");
    }
}

class Car {
    private Engine engine;

    public Car() {
        engine = new Engine(); // Car is tightly coupled with Engine
    }

    public void startCar() {
        engine.start();
        System.out.println("Car is starting...");
    }

    public static void main(String[] args) {
        Car car = new Car();
        car.startCar();
    }
}
```

Loosely Coupled Components:

// Loosely Coupled Example

```
interface Engine {
    void start();
}
```

```

}

class PetrolEngine implements Engine {
    public void start() {
        System.out.println("Petrol Engine is starting...");
    }
}

class DieselEngine implements Engine {
    public void start() {
        System.out.println("Diesel Engine is starting...");
    }
}

class Car {
    private Engine engine;

    public Car(Engine engine) { // Engine is passed as a dependency
        this.engine = engine; // Car is loosely coupled with Engine
    }

    public void startCar() {
        engine.start();
        System.out.println("Car is starting...");
    }

    public static void main(String[] args) {
        Engine petrolEngine = new PetrolEngine();
        Car car = new Car(petrolEngine);
        car.startCar();

        Engine dieselEngine = new DieselEngine();
        Car car2 = new Car(dieselEngine);
        car2.startCar();
    }
}

```

Dependency injection

17 August 2024 15:14

- ▶ When we are creating an object we have to manage the entire cycle of object i.e, manging , creation. We create multiple objects unnecessarily
 - ▶ loc: we create the object ourself, some one else create our object called Inversion of control.
 - ▶ In spring to achieve ioc we use Dependency injection
 - ▶ loc is a principle and dependency is the design pattern
 - ▶ Dependency injection is the implementation of loc.
 - ▶ Whenever we want an object we will request spring it **injects** the object
 - Dependency is a design pattern for software development projects in which object dependencies are provided externally rather than creating itself.
 - Dependency injection is useful for creating loosely coupled programs.
- Advantages
- It is easy to test and maintain
 - Makes the code loosely coupled

Note:-

- Dependency injection is used to remove dependencies from program

Types of dependency injection

- Setter injection
- Constructor injection
- ▶ Field injection

Setter injection

- Generating setters and getters method in POJO class

Constructor injection

- Generating default constructor in POJO class

Inversion Of control

17 August 2024 15:21

IOC is a design pattern for software development process where it is responsible to inject dependencies.

- Different types of IOC containers are
 - o BeanFactory
 - o Application Context
- Example:
 1. Setter injection using Bean factory and application context
 2. Setter injection using application context
 3. Constructor Injection using beanFactory
 4. Constructor Injection using application context
- Note: in spring framework we must have the following files POJO class / Bean Class
- Configuration File
- Logic file

Note:- syntax for using bean factory

```
Resource r = ClassPathResource("config.xml");  
BeanFactory bf = XmlBeanFactory(r);
```

Note:- Syntax for application context

```
ApplicationContext ac = ClassPathXmlApplicationContext(config.xml);
```

SI using BF

19 August 2024 11:23

- Note:-

- create maven project
- Update maven compiler from 1.7 to 1.8
- add dependencies
- update maven project
- Create resources under src/main
- Implement Spring{Bean class, configuration file, logic file}
- Run--> Java application

i.

- a. Bean class(employee.java)

```
Class employee{
```

```
    Int eid;
```

```
    String ename;
```

```
    Double esal;
```

```
    // setter injection --> generate getters and setters
```

```
}
```

- b. Configuration file(spconfig.xml)

```
Spring dtd
```

```
<beans>
```

```
    <bean id="empbean" name="Packagename.projectName"
        class="Package.Project.class">
```

```
        <Property name="eid" value="111"/>
```

```
        <Property name="ename" value="abc"/>
```

```
        <Property name="esal" value="4999"/>
```

```
    </bean>
```

```
</beans>
```

- c. logic file

```
Test.java
```

```
Class Test{
```

```
    Public static void main(String [] args){
```

```
        Resource r = ClassPathResource("config.xml");
```

```
        BeanFactory bf = XmlBeanFactory(r);
```

```
        Object Obj=Bf.getBean("empbean");
```

```
Employee e = (Employee)obj;  
System.out.println(e.getId()+""+e.getName()+""+e.getEsal());  
  
}  
  
}
```


SI using AC

19 August 2024 11:51

a. Bean class(employee.java)

Class employee{

Int eid;

String ename;

Double esal;

// setter injection --> generate getters and setters

}

b. Configuration file(spconfig.xml)

Spring dtd

<beans>

<bean id="empbean" name="Packagename.projectName" class="Package.Project.class">

<Property name="eid" value="111"/>

<Property name="ename" value="abc"/>

<Property name="esal" value="4999"/>

</bean>

</beans>

c. logic file

ApplicationTest.java

Class ApplicationoTest{

Public static void main(String [] args){

ApplicationContext ac = ClassPathXmlApplication("config.xml");

Object Obj=ac.getBean("empbean");

Employee e = (Employee)obj;

System.out.println(e.getId()+" "+e.getName()+" "+e.getEsal());

}

}

ii. Another type

a. Bean class(employee.java)

Class employee{

Int eid;

String ename;

```
Double esal;
```

```
// setter injection --> generate getters and setters
```

```
Void display(){  
    System.out.println(Ename);  
}
```

```
}
```

b. Configuration file(spconfig.xml)

Spring dtd

```
<beans>
```

```
    <bean id="empbean" name="Packagename.projectName" class="Package.Project.class">
```

```
        <Property name="eid" value="111"/>
```

```
        <Property name="ename" value="abc"/>
```

```
        <Property name="esal" value="4999"/>
```

```
    </bean>
```

```
</beans>
```

c. logic file

Test.java

```
Class Test{
```

```
    Public static void main(String [] args){
```

```
        ApplicationContext ac = ClassPathXmlApplicationContext("config.xml");
```

```
        Object Obj=ac.getBean("empbean");
```

```
        Employee e = (Employee)obj;
```

```
        e.display
```

```
    }
```

```
}
```

CI using BF

19 August 2024 11:56

i. Another type

a. Bean class(Employee.java)

```
Class employee{
    Int eid;
    String ename;
    Double esal;
    //generator getter method
    Public employee(int eid,String ename,Double esal){
        This.eid=eid;
        This.ename=ename;
        This.esal=esal;
    }
}
```

b. Configuration file(spconfig.xml)

```
Spring dtd
<beans>
    <bean id="empbean" name="Packagename.projectName"
        class="Package.Project.class">

        <constructor-arg value="222" type="int"/>
        <constructor-arg value="def" type="String"/>
        <constructor-arg value="3500" type="double"/>

    </bean>

</beans>
```

c. logic file

```
Test.java
Class Test{
    Public static void main(String [] args){

        Ex-1}

    }
```

CI using AC

19 August 2024

12:12

i.

a. Bean class(Employee.java)

```
Class employee{
    Int eid;
    String ename;
    Double esal;
    //generator getter method
    Public employee(int eid,String ename,Double esal){
        This.eid=eid;
        This.ename=ename;
        This.esal=esal;
    }
}
```

b. Configuration file(spconfig.xml)

```
Spring dtd
<beans>
    <bean id="empbean" name="Packagename.projectName"
        class="Package.Project.class">

        <constructor-arg value="222" type="int"/>
        <constructor-arg value="def" type="String"/>
        <constructor-arg value="3500" type="double"/>

    </bean>

</beans>
```

c. logic file

```
Test.java
Class Test{
    Public static void main(String [] args){

        Ex-2
    }

}
```

Data types

20 August 2024 11:32

- The purpose of primitive and non primitive data types are to know what kind of data a bean object be hold(or) what kind of data can be injected to a bean.
- The different types of data which can be hold by a bean object are

Primitive data type:

- A bean object will hold primitive values if we implement bean class using setters and getters
- Class using setters and getters
- Example

Bean class

->Employee

--generate getters and setters

Configuration file

```
<beans>
  <bean id="empbean" class="packageName.className">
    <property name="" value="">
  </bean>
```

```
</beans>
```

Logic file:-

```
ApplicationContext ac = new ClassPathXmlApplicationContext("spconfig.xml");
```

```
Object obj = ac.getBean("empbean");
```

```
Employee e = (Employee ) obj;
```

```
System.out.println(e.getId());
```

Non Primitive data type: a bean object will hold either collections or object will refer to non-primitive non primitive data types or non-primitive values.

- A bean object will hold non- primitive values if we implement bean class using constructor
- Collections: constructor

- Three types

□ List:- collection

a) Question.java

```
package com.nonPrimitive;

import java.util.Iterator;
import java.util.List;

public class Question {
    int qid;
    String qname;
    List<String> answers;

    @SuppressWarnings("unchecked")
    public Question(int qid,String qname, List answers) {
        this.qid=qid;
        this.qname = qname;
        this.answers = answers;
    }
    void display() {
        System.out.println("the answers are");
        Iterator itr = answers.iterator();
        while(itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}

b) Spconfig.xml
<bean id="qbean" class="com.nonPrimitive.Question">

  <constructor-arg value="101" type="int" />
  <constructor-arg value="what is your favourite color?" type="String" />
  <constructor-arg>
    <list>
      <value>Blue</value>
      <value>Black</value>
      <value>Red</value>
      <value>Yellow</value>
    </list>
```

```

</constructor-arg>
</bean>

```

c)

ListN.java

```

package com.nonPrimitive;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class ListN {
    public static void main(String[] args) {
        ApplicationContext ac = new ClassPathXmlApplicationContext("spconfig.xml");
        Object obj = ac.getBean("qbean");
        Question q = (Question) obj;
        q.display();
    }
}

```

□ Set: It is a collection of items

```

package com.nonPrimitive;

```

```

import java.util.Iterator;
import java.util.List;

```

```

public class Question {
    int qid;
    String qname;
    List<String> answers;

    @SuppressWarnings("unchecked")
    public Question(int qid, String qname, List answers) {
        this.qid = qid;
        this.qname = qname;
        this.answers = answers;
    }
    void display() {
        System.out.println("the answers are");
        Iterator itr = answers.iterator();
        while(itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}

```

a) Spconfig.xml

```

<bean id="qbean" class="com.nonPrimitive.Question">

    <constructor-arg value="101" type="int" />
    <constructor-arg value="what is your favourite color?" type="String" />
    <constructor-arg>
    <Set>
    <value>Blue</value>
    <value>Black</value>
    <value>Red</value>
    <value>Yellow</value>
    </Set>
    </constructor-arg>
</bean>

```

a)

SetN.java

```

package com.nonPrimitive;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class ListN {
    public static void main(String[] args) {
        ApplicationContext ac = new ClassPathXmlApplicationContext("spconfig.xml");
        Object obj = ac.getBean("qbean");
        Question q = (Question) obj;
        q.display();
    }
}

```

◇ Map

- a. Object : setter getter, the kind of data which is going to be hold by an bean object is called object, if the bean class implemented using setter injection.

- Example:- library class(Auto wiring)

- o Bean class- Book.java

```
package com.Object;

public class Book {
    int bid;
    String bauthor;
    int bcost;
    //generate setters
    public int getBid() {
        return bid;
    }
    public void setBid(int bid) {
        this.bid = bid;
    }
    public String getBauthor() {
        return bauthor;
    }
    public void setBauthor(String bauthor) {
        this.bauthor = bauthor;
    }
    public int getBcost() {
        return bcost;
    }
    public void setBcost(int bcost) {
        this.bcost = bcost;
    }
}
```

- o Lib bean

```
package com.Object;

public class Library {
    Book lbook;
    String lname;
    //getters and setters
    public Book getLbook() {
        return lbook;
    }
    public void setLbook(Book lbook) {
        this.lbook = lbook;
    }
    public String getLname() {
        return lname;
    }
    public void setLname(String lname) {
        this.lname = lname;
    }
}
```

- Spconfig.xml

```
<bean id="bookbean" class="com.Object.Book">
<property name="bid" value="111"/>
<property name="bauthor" value="rs agarwal"/>
<property name="bcost" value="399"/>
</bean>
<bean id="Libbean" class="com.Object.Library">
<property name="lbook" ref="bookbean"/>
<property name="lname" value="center library"/>
</bean>
```

- Lib test

```
public static void main(String[] args) {
    ApplicationContext ac = new ClassPathXmlApplicationContext("spconfig.xml");
    Object obj = ac.getBean("libbean");
    Library lib = (Library)obj;
    System.out.println("The book bean attributes are\n book "+lib.lbook.getBid()+"\n library name:"+lib.getLname());
}
```


DAO Hibernate

26 August 2024 11:51

Data access object: in this concept we will understand how to integrate hibernate application or framework with spring application

Hibernate:- hibernate.cfg.xml

Spring:- applicationContext.xml

- In hibernate application, we provide all the database application in a file named as hibernate.cfg.xml(configuration file) whereas in the case of spring framework we provide all the database information in a file named as applicationContext.xml(configuration file)
- The advantages of spring with hibernate
 - o Spring provides a new class named as hibernateTemplate which saves a lot of codes to interact with database
 - o Hibernate is based on ORM whereas spring is based on DAO
 - o The components of spring DAO/ Framework are(spring example to interact with database must have the following files)
 - Bean class
 - Mapping files
 - Interface
 - Implementation class
 - Configuration file
 - Logic file

- o Example: SpringDao with hibernate

Note:

S-1: maven project

S-2: update maven compiler

Add dependencies- (spring-core, spring-context, spring-bean)
(Hibernate ORM, Spring ORM

S-3: update maven project

S-4: create directory---- resource for [configuration&mapping file]

S-5: Bean class

S-6: mapping file

S-7: create interface

S-8: implementation class

S-9: configuration file

S-10: logic file

S-2: dependencies

```
<dependencies>
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-core</artifactId>
<version>5.3.8</version>
</dependency>

<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-beans</artifactId>
<version>5.3.8</version>
</dependency>

<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>5.3.8</version>
</dependency>

<!-- Spring ORM (For Hibernate Integration) -->
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-orm</artifactId>
<version>5.3.8</version>
</dependency>

<!-- Hibernate ORM -->
<dependency>
<groupId>org.hibernate</groupId>
<artifactId>hibernate-core</artifactId>
<version>5.4.31.Final</version>
</dependency>

<!-- JDBC Driver (Assuming you are using MySQL, change if using another
database) -->
<dependency>
```

```

<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>8.0.26</version>
</dependency>

<!-- C3P0 (If you use it for connection pooling) -->
<dependency>
<groupId>com.mchange</groupId>
<artifactId>c3p0</artifactId>
<version>0.9.5.5</version>
</dependency>

<!-- Spring Transactions -->
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-tx</artifactId>
<version>5.3.8</version>
</dependency>

<dependency>
<groupId>com.mysql</groupId>
<artifactId>mysql-connector-j</artifactId>
<version>8.1.0</version>
</dependency>
<dependency>
<groupId>commons-dbcp</groupId>
<artifactId>commons-dbcp</artifactId>
<version>1.4</version> <!-- or the version you prefer -->
</dependency>
</dependencies>

```

configuration file- applicationContext.xml

```

<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xmlns:tx="http://www.springframework.org/schema/tx"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx.xsd">
<bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource">
<property name="driverClassName" value="com.mysql.cj.jdbc.Driver"></property>
<property name="url" value="jdbc:mysql://localhost:3306/springDao"></property>
<property name="username" value="root"></property>
<property name="password" value="admin"></property>
</bean>

<bean id="mySessionFactory" class="org.springframework.orm.hibernate5.LocalSessionFactoryBean">
<property name="dataSource" ref="dataSource"></property>
<property name="mappingResources">
<list>
<value>employee.hbm.xml</value>
</list>
</property>
<property name="hibernateProperties">
<props>
<prop key="hibernate.dialect">org.hibernate.dialect.MySQL5Dialect</prop>
<prop key="hibernate.hbm2ddl.auto">update</prop>
<prop key="hibernate.show_sql">true</prop>
</props>
</property>
</bean>
<bean id="ht" class="org.springframework.orm.hibernate5.HibernateTemplate">
<property name="sessionFactory" ref="mySessionFactory"></property>
</bean>
<bean id="empdao" class="com.springDao.hibernate.EmployeeRemote">
<property name="ht" ref="ht"/>
</bean>
<bean id="transactionManager" class="org.springframework.orm.hibernate5.HibernateTransactionManager">
<property name="sessionFactory" ref="mySessionFactory"/>
</bean>
<tx:annotation-driven transaction-manager="transactionManager"/>
</beans>
Employee.hbm.xml

```

```

<hibernate-mapping>
<class name="com.springDao.hibernate.Employee" >
<id name="eid"/>
<property name="ename"/>
<property name="esal"/>
</class>

```

```
</hibernate-mapping>
```

```

BEAN CLASS:-
Employee.java

```

```
package com.springDao.hibernate;
```

```
package com.springDao.hibernate;
```

```
public class Employee {  
  
    int eid;  
    String ename;  
    double esal;  
    public int getEid() {  
        return eid;  
    }  
    public void setEid(int eid) {  
        this.eid = eid;  
    }  
    public String getEname() {  
        return ename;  
    }  
    public void setName(String ename) {  
        this.ename = ename;  
    }  
    public double getEsal() {  
        return esal;  
    }  
    public void setEsal(double esal) {  
        this.esal = esal;  
    }  
    public String toString() {  
        return "Employee{" +  
            "id=" + eid +  
            ", name='" + ename + '\'' +  
            ", salary=" + esal +  
            '\'';  
    }  
}
```

```
}
```

INTERFACE:-

EmployeeDao.java

```
package com.springDao.hibernate;
```

```
import java.util.List;
```

```
public interface EmployeeDao {  
    void insertEmployee(Employee emp);  
    void updateEmployee(Employee emp);  
    void deleteEmployee(Employee emp);  
    Employee findEmployeeById(int eid);  
    List<Employee> getEmployees();  
}
```

```
}
```

EmployeeRemote.java

```
package com.springDao.hibernate;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
import org.springframework.orm.hibernate5.HibernateTemplate;  
import org.springframework.transaction.annotation.Transactional;
```

```
public class EmployeeRemote implements EmployeeDao {  
    HibernateTemplate ht;
```

```
    public void setHt(HibernateTemplate ht) {  
        this.ht = ht;  
    }  
}
```

```
@Override
```

```
@Transactional
```

```
public void insertEmployee(Employee emp) {  
    ht.save(emp);  
}
```

```
}
```

```
@Override
```

```
@Transactional
```

```
public void updateEmployee(Employee emp) {  
    ht.update(emp);  
}
```

```

    }

    @Override
    @Transactional
    public void deleteEmployee(Employee emp) {
        ht.delete(emp);
    }

    @Override
    public Employee findEmployeeById(int eid) {
        Employee emp1 = ht.get(Employee.class, eid);
        return emp1;
    }

    @Override
    public List<Employee> getEmployees() {
        //
        //      List<Employee> empl = new ArrayList<>();
        //      empl.add((Employee) ht.loadAll(Employee.class));
        //      return empl;
        //      return ht.loadAll(Employee.class);
    }
}

```

EmployeeTest.java- logic file

```

package com.springDao.hibernate;

import java.util.List;
import java.util.Scanner;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class EmployeeTest {

    public static void main(String[] args) {
        @SuppressWarnings("resource")
        ApplicationContext ac = new ClassPathXmlApplicationContext("applicationContext.xml");

        EmployeeDao dao = (EmployeeDao) ac.getBean("empdao");
        Employee emp3 = new Employee();

        //      emp3.setEid(121);
        //      emp3.setEname("ABC");
        //      emp3.setEsal(16661.14);
        //

        Scanner sc = new Scanner(System.in);
        System.out.println("Enter your desired choice");

        int id;
        while (true) {
            System.out.println("1.insert\n2.delete\n3.update\n4.view by id\n5.view all\n0.enter 0 for exit");
            int choice = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.println("enter emp id");
                    id = sc.nextInt();
                    System.out.println("enter employee name");
                    String name = sc.next();
                    System.out.println("enter employee salary");
                    double sal = sc.nextDouble();
                    if (isInsert(id, name, sal, ac))
                        System.out.println("inserted successfully");
                    else
                        System.out.println("employee id already exists");
                    break;

                case 2:
                    System.out.println("enter id to delete");
                    id = sc.nextInt();
                    if (isDelete(id, ac))
                        System.out.println("deleted successfully");
                    else
                        System.out.println("employee id doesnt exists");
                    break;

                case 3:
                    System.out.println("enter id to update");
                    id = sc.nextInt();
                    if (isUpdate(id, ac))
                        System.out.println("updated successfully");
                    else
                        System.out.println("employee id doesnt exists");
                    break;

                case 4:

```

```

        System.out.println("enter id to view");
        id = sc.nextInt();
        Employee emp = dao.findEmployeeById(id);
        if (emp != null) {
            System.out.println(emp.toString());
        } else {
            System.out.println("Employee not found.");
        }
        break;
    case 5:
        List<Employee> employees = dao.getEmployees();
        if (employees.isEmpty()) {
            System.out.println("No employees found.");
        } else {
            for (Employee emp12 : employees) {
                System.out.println(emp12.toString());
            }
        }
        break;
    default:
        System.exit(0);
    }

}

// dao.insertEmployee(emp3);

}

public static boolean isInsert(int id, String name, double sal, ApplicationContext ac) {
    EmployeeDao daoi = (EmployeeDao) ac.getBean("empdao");
    Employee emp4 = new Employee();
    emp4.setEid(id);
    emp4.setName(name);
    emp4.setEsal(sal);
    try {
        daoi.insertEmployee(emp4);
        return true;
    } catch (Exception e) {

        return false;
    }

}

public static boolean isDelete(int id, ApplicationContext ac) {
    EmployeeDao daod = (EmployeeDao) ac.getBean("empdao");

    try {
        Employee emp5 = daod.findEmployeeById(id);
        if (emp5 != null) {
            daod.deleteEmployee(emp5);
            return true;
        } else {
            return false;
        }
    } catch (Exception e) {
        return false; // Error occurred during delete operation
    }

}

public static boolean isUpdate(int id, ApplicationContext ac) {
    EmployeeDao daou = (EmployeeDao) ac.getBean("empdao");
    Employee emp6 = new Employee();
    Scanner sc = new Scanner(System.in);
    emp6.setEid(id);
    System.out.println("enter name edit");
    String name = sc.next();
    emp6.setName(name);
    System.out.println("enter salary to edit");
    double sal = sc.nextDouble();
    emp6.setEsal(sal);
    try {
        daou.updateEmployee(emp6);
        return true;
    } catch (Exception e) {
        return false;
    }

}

}

```

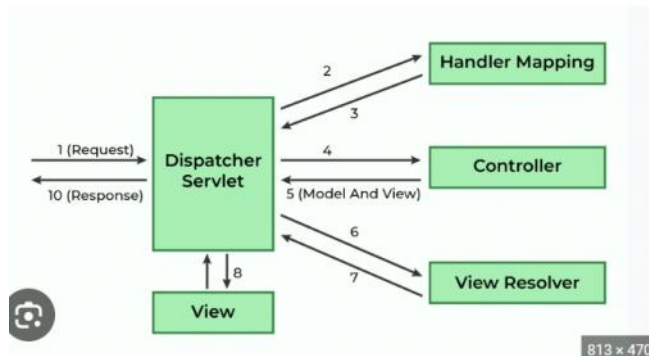
Spring JDBC

21 September 2024 13:49

Spring MVC

23 September 2024 11:25

- MVC: model view and controller
 - o Model : business logic / application data / code (back-end)
 - o View: Presentation logic / User interface (front-end)
 - o Controller : it acts as interface between model and view
 - Interface between business and presentation logic
 - Inter face between frontend and backend
- MVC is one of the popular and standard design pattern/architecture which will be used by most of the developers to build web application
- Spring MVC: it is a java framework (part of spring framework) which is used to design web applications
- Spring MVC includes all the basic core components of spring like dependency injection and inversion of control.
- Spring framework provides an elegant solution to build web application following MVC architecture using DispatcherServlet
- The architecture of spring MVC is



- Dispatcher servlet is class in spring framework that receives an Http Requests and maps the right resources such as either it has mapped with model view controller.
- Steps involved in spring mvc
 - i. Http servlet
 - ii. Dispatcher servlet
 - iii. Handler mapping (url pattern/mapping)
 - iv. Controller (model and view)
 - v. View Resolver
 - vi. View (http response)
- Explanation :
 - o This architecture follows sequence of events corresponding to an incoming http request to dispatcher Servlet
 - o After receiving HttpRequests, DispatcherServlet consults handler mapping to call the appropriate controller
 - o The controller will take the request and call the appropriate service methods using GET and POST methods
 - o Then the DispatcherServlet will take the help of ViewResolver to pickup the right view based on HttpRequest
 - o Finally, the view which got picked by viewResolver will be HttpResponse
- Advantages of spring MVC
 - o Light weight
 - o Rapid development
 - o Separate roles
 - o Re-usability of a code
 - o Easy to test
- Example: spring WEB-MVC
 - Step-1: maven-archetype-webapp : 1.4
 - Step-2: Update maven compiler
 - Step-3: update maven project
 - Step-4 : resource directory (config files and mapping files)
 - Step-5: implementation of spring-mvc
 - Dispatcher servlet
 - View Resolver
 - Step-6 : need to run server (apache tomcat)
 - Step-7 : run this application (run as server)

@Component: create Object for the particular class by spring boot

@Autowired: connecting two classes which are dependent each other

@RestController

@Controller

@ComponentScan

@

Introduction

15 October 2024 11:16

- Introduction
- Why SpringBoot
- Features of Spring Boot
- Advantages of Spring Boot
- Spring Boot Architecture Working with Spring Boot (3 ways)
 - Spring Initializr
 - SAS Tool (IDE)
 - CLI (Command Line interface) -
- Spring Boot examples (SI)
- Thymeleaf and views(html/jsp) rendering - Spring Starters (web) - Spring Boot web MVC(CRUD) - Spring Boot Security

Spring initializer

15 October 2024 11:35

1. Spring initializer

Step-1: Open [Spring InitializEr](#)

Step-2: maven-->java-->spring boot version (latest without snapshot)

- Project meta data(jar)
- Jar--java17
- Dependency
 - Spring web
 - Spring security
 - Spring DevTool
 - Spring thymeleaf
- generate

Step-3:

Spring boot examples:

Example-1: to display output on console	
Example-2: to display a string in .java on A browser	@RestController
Example-3: to render a html file in templates with With thymeleaf(only html) or without	@Controller
Example-4: to render a jsp file	
Example-5: to render a form using jsp	
Example-6: render a form when u click on submit button display message on the browser as details submitted successfully	
Example-7: render a form when u click on submit button display details in the form on a webpage	@RequestParam
Example-8: provide path name in URL and display the same on the web page	@PathParam
Example-9: addition	@PathVariable

Note: to return a string we use @RestController and for html @Controller

** thyme leaf views rendering

Of html or jsp page

Example -3: to render a view(html)

```
//controller2.java
@Controller //for html rendering
public class SampleController {

    @RequestMapping("/sample")
    public String method2() {
        return "index";
    }

}
```

Note: to render an html view my controller must be defined using @Controller

- Html files must be placed in template, with thymeleaf dependency
- Else we can place in static

Example-4:

```
package com.klef.springboot.SPINGBOOTDEMO;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

//controller2.java
@Controller //for jsp rendering
public class ControllerDemo {

    @GetMapping("/jsp")
    public String method2() {
        return "new";
    }
}
```

Note:

Spring boot application/project components

1. Application properties

- It is a component which defines all the properties of a spring boot applications
- Src/main/resources ---> application.properties
- Ex:- spring.application.name = project name
- Server.port = #PORT_NUMBER
- Spring.mvc.view.prefix = /views/
- Spring.mvc.view.suffix = .jsp

2. Annotations

- Annotation is a form of data about application.

a. Spring core annotations

- @Required :-
 - This is method level annotation
 - This annotation is used for method which is mandatory to execute
- @ComponentScan :-
 - This is class level annotation
 - It is used to scan a package for beans
- @Bean :-
 - This is method level annotation
 - It is an alternative to an xml file
 - It tells the method to produce a bean managed by a spring container.
- @Component
 - It is a class level annotation
 - It is used to mark a java class as a bean
- @service
 - Class level annotation
 - It is used where the business logic of the application is defined
- @Repository
 - This is class level annotation
 - It is used for a class where DAO

b. Spring boot annotations

- @SpringBootApplication :-
 - Class level annotation

- It represents starting point of application
- This annotation is a combination of 3 annotations
 - ◆ @EnableAutoConfiguration
 - ◆ @ComponentScan
 - ◆ @Configuration
- ii. @RestController : -
 - This is class level annotations
 - REST services
- iii. @RequestMapping : -
 - This is method level annotation
 - It is used for any kind of http Request.
- iv. @GetMapping : -
 - This is method level annotation
 - It maps an HTTP GET request
- v. @PostMapping : -
 - This is method level annotation
 - It maps HTTP POST request.
- vi. @PUTMapping:
 - Method level
 - It maps http PUT request
- vii. @DeleteMapping : -
 - This is method level annotation
 - It is used for http DELETE Request
- viii. @Controller : -
 - Class level annotation
 - It is used for class where the method are defined to render view
- 3. Dependencies management
 - It manages all the dependencies for maven project (using SI)
 - The dependencies will be managed in file pom.xml file
- 4. Web-app :
 - It manages all the views to be rendered.
 - It must be created by developer
- 5. Spring starters

Spring boot

21 October 2024 11:46

Spring boot application/project components

1. Application properties

- It is a component which defines all the properties of a spring boot applications
- Src/main/resources ---> application.properties
- Ex:- spring.application.name = project name
- Server.port = #PORT_NUMBER
- Spring.mvc.view.prefix = /views/
- Spring.mvc.view.suffix = .jsp

2. Annotations

- Annotation is a form of data about application.

a. Spring core annotations

i. @Required :-

- This is method level annotation
- This annotation is used for method which is mandatory to execute

ii. @ComponentScan :-

- This is class level annotation
- It is used to scan a package for beans

iii. @Bean :-

- This is method level annotation
- It is an alternative to an xml file
- It tells the method to produce a bean managed by a spring container.

iv. @Component

- It is a class level annotation
- It is used to mark a java class as a bean

v. @service

- Class level annotation
- It is used where the business logic of the application is defined

vi. @Repository

- This is class level annotation
- It is used for a class where DAO

b. Spring boot annotations

i. @SpringBootApplication :-

- Class level annotation
- It represents starting point of application
- This annotation is a combination of 3 annotations
 - ◆ @EnableAutoConfiguration

◆ @ComponentScan

◆ @Configuration

ii. @RestController : -

- This is class level annotations
- REST services

iii. @RequestMapping : -

- This is method level annotation
- It is used for any kind of http Request.

iv. @GetMapping : -

- This is method level annotation
- It maps an HTTP GET request

v. @PostMapping : -

- This is method level annotation
- It maps HTTP POST request.

vi. @PUTMapping:

- Method level
- It maps http PUT request

vii. @DeleteMapping : -

- This is method level annotation
- It is used for http DELETE Request

viii. @Controller : -

- Class level annotation
- It is used for class where the method are defined to render view

3. Dependencies management

- It manages all the dependencies for maven project (using SI)
- The dependencies will be managed in file pom.xml file

4. Web-app :

- It manages all the views to be rendered.
- It must be created by developer

5. Spring starters

Spring starters

21 October 2024 12:19

Starters: it is collection of dependencies which simplifies the development process of spring boot application

- The different spring starters are
 - Spring web :- this starter is used to build web applications using spring boot
Ex: any application that can be run on localhost that runs if "web " starter existence.
 - Spring parent:
 - it is used in maven project to simplify dependency management and configuration for spring applications.
 - It acts as project object model (POM) that manages the versions of dependencies and plugins to support the applications
 - Spring actuator:
 - This starter provides production ready features for monitoring and managing your application which includes (example : health check---{performance of application})
 - Spring test
 - This starter provides set of testing tools to test spring boot application
 - It provides JUNIT testing tool
 - Spring JPA: [click](#)
 - Spring security:
 - This starter provide security to your spring boot application providing authentication and authorization as the key features
 - Spring thymleaf:
 - This starter integrates thymeleaf engine with spring boot allowing you to create dynamic web application.
 - Spring dev tools :
 - It enhances the development experience in spring boot applications by providing features like automatic restart, live reload, debugging, etc..

Example

26 October 2024 13:55

Ex-5: to render a form using jsp

Ex-6: render a form when u click on submit button display message on the browser as details submitted successfully

Ex-7: render a form when u click on submit button display details in the form on a webpage

@GetMapping("/formdetails")

```
public String details(@RequestParam("fn") String name, @RequestParam("phno") String mobile ,Model model) {  
    model.addAttribute("name", name);  
    model.addAttribute("mobile", mobile);  
    return "formdetails";  
}
```

Name: \${name}

Mobile: \${mobile}

<form action="formdetails" >

Name:<input type="text" name="fn">

Mobile Number:<input type="number" name="phno">

<input type="submit" value="Submit">

Ex-8: provide path name in URL and display the same on the web page

@GetMapping("/input/{name}")

```
public String path(@PathVariable("name") String name,Model model) {  
    model.addAttribute("name",name);  
    return "display";  
}
```

Ex-9: addition of nummmbers

Input: 2 path variables - a,b;

Output: sum

Localhost:8080/input/5/10

```
Public String add(@PathVariable("a") int a, @PathVariable("b") int b, Model model){  
    Sum = a+b;  
    Model.addAttribute(attributename:"sum",sum)  
}
```

Ex-10:

Spring starter JPA:

- It is a dependency in spring boot which provides database access
- Example: spring boot application to interact with data base using Command Line Runner and entity manager class using MySQL
- Project--
 - Starter
 - Dependencies : devtools, jpa,
 - Testing: print("working...") to check weather our application works fine or not
 - Application.properties
 - Server.port
 - Spring.datasource.url
 - Spring.datasource.database
 - Spring.datasource.password
 - Spring.datasource.ddl-auto=update // no need to use annotations in entity class**

- Entity
 - Class Employee{
 - @ID
 - Private int eid;
 - Private string ename;
 - Private double esal;

Setters and getters
}

- Create an interface

```
package com.klu.service;

import java.util.List;

import com.klu.entity.Employee;

public interface EmployeeDao {
    public void insertEmployee(Employee e);
    public void deleteEmployee(int eid);
    public void updateEmployee(int eid);
    public List<Employee> viewAll();
    public Employee viewById(int eid);
}
```

- Implementations class

```
package com.klu.service;
```

```

import java.util.List;
import com.klu.entity.Employee;
import jakarta.persistence.EntityManager;
public class EmployeeDaoImp implements EmployeeDao{
    EntityManager em;
    @Override
    public void insertEmployee(Employee e) {
        em.persist(e);

    }

    @Override
    public void deleteEmployee(int eid) {

    }

    @Override
    public void updateEmployee(int eid) {
        // TODO Auto-generated method stub

    }

    @Override
    public List<Employee> viewAll() {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public Employee viewById(int eid) {
        // TODO Auto-generated method stub
        return null;
    }

}

```

- Load objects using Command line runner
 - EmployeeLoad.java
 - Public class EmployeeLoad implements CommandLineRunner

SBMVC

05 November 2024 11:19

Mvc: model view controller, it is standard design pattern which will be used by many of the developers to build a web application.

Model: represents business logic of our application.(Backend)

View: it represents the user interface of our application

Controller: it acts as interface between model and view

Example:

Spring boot application to interact with a database using mvc architecture (using JPA repository)

---> start.spring.io-->maven--> version-->group and Artifact --> dependencies

SB security

05 November 2024

12:45

- It is a starter in spring boot framework which provides security to our application
- It provides basic authentication and authorization to spring boot application
- Example: spring boot application to provide authentication and authorization
- Note: auto wiring in spring and spring boot
- Dependencies: web,devTools, security

```
spring.security.user.name = system  
spring.security.user.password = admin
```

-

Micro services

09 November 2024 15:23

Spring cloud and micro services:

Monolithic: - it is a single application that will be developed as single application with multiple services

ERP Project contains 3 roles:

- 1.admin
- 2.faculty
- 3.student

Disadvantages:

- It is harder to implement changes in larger and complex application
- Changing technology or changing any full stack of entire project is difficult
- It will become too complicated to understand and to manage the code

Tight coupling architecture vs loose coupling architecture

What is microservice?

It contains single service

- instead of incorporating more than one service in one single project, we are going to develop project for every single/ individual service.
- Small, independent and loosely coupled
- Each service will have separate code base
- Services can be deployed independently
- Services can be communicated by exchanging the data with each other by using Rest Template
- Services no need to share same technology stack
- ADMIN microservice {spring boot mvc/ REST api} 2000- /admin/**
- STUDENT microservice {Django} 2002 /student/**
- FACULTY microservice {node/ express} 5000 /faculty/**

Spring cloud

10 November 2024

10:16

Spring cloud: it is a collection of tools for developing micro services based applications on JVM platform. It provides a set of libraries and tools that make it easier to build and deploy microservices especially those that are deployed in cloud environment.

- Advantages
 - It allows one service to communicate with another service
 - It is used to develop applications based on the cloud
 - Its architecture is designed according to the micro service
- Develop and deploy microservice based applications on cloud:
- EUREKA...
 - It is a service discovery tool for microservice architecture developed by Netflix (saas).
 - It provides a registry where services can register themselves and discover other registered services
 - Total 3 services- register and discover
 - Key features/components:
 - i. Server register {service registry}: services can register themselves with eureka server providing their location and meta data
 - ii. Server discovery:
Clients can query server to discover the available instances of a particular service. We are going to find specific instances based on port{4000,4001,4002} instead of port we use application name and hostname
 - iii. Load balancing :
1000 users at a time want to access student service? Eureka supports client-side load balancing allowing clients to distribute requests across multiple instances of a service.
 - iv. Fault tolerance:
Eureka can detect and remove failed instances from the registry ensuring that clients are not directed to unavailable instances.
 - Add starter EUREKA SERVER
 - Default port of eureka is 8761
 - @EnableEurekaServer
 - AdminService [Eureka client 1]
 - FacultyService[Eureka client 2]
 - We set this property to false so that it does not register itself in the list along with other clients. Because, here it acts as a server but not as client

```
eureka.client.register-with-eureka= false
```

- The value is set to false because it will not retrieve the registered microservices(eureka clients) list from anywhere. It will create and maintain this list

```
eureka.client.fetch-registry = false
```

- Note: eureka server and clients will be grouped by hostnames(by default)
- Server should be started first

- API gateway

- If one or more microservices running on different port numbers. API gateway, will have only one port number to redirect to different microservices.
- Route: it is a basic building block of gateway, it consists of ID(Application/server name)
 - ID
 - Destination: URI
 - Collection of predicates and collection of filters
 - Predicate: it is similar to JAVA 8 function predicate. Using this, we can match HTTP request such as header, URL, cookies or parameters
 - Filter: these are the instances of spring framework gateway filter. Using this, we can modify the request or response as per the requirement
 - Note: predicate is an object that will test if the given request satisfies a given condition or not. For each route, we can define one or more predicates and if satisfied, will accept the request.
 - EUREKA server
 - Total 3 eureka clients
 - ◆ Index1 -admin
 - ◆ Index2- faculty
 - ◆ Index3-student
 - Instead of accessing individual microservice project on different port numbers, now I am accessing with single port number called as 2000 using API gateway by redirecting based on path

Cloud

12 November 2024

11:18

Spring cloud and micro services:

Spring cloud:

- Introduction
- Why spring cloud
- Features of spring cloud
- Advantages of spring cloud
- Architecture of spring cloud
- Spring cloud components
- Main projects of spring cloud
- Spring boot VS spring Cloud

Microservices

- Introduction
- Why microservices
- Features of microservices
- Advantages of micro services
- Architecture of micro services{MSA- micro service architecture: S&A- service oriented architecture }
- Ex-1:--Creating simple micro service

Spring cloud

12 November 2024 11:31

- it is a framework for building cloud application
- Spring cloud is also developed by dave syer
- it is a collection of tools for developing micro services based applications on JVM platform. It provides a set of libraries and tools that make it easier to build and deploy microservices especially those that are deployed in cloud environment.
- It is used to simplify microservices.

Why spring cloud:

1. Configuration management: Centralized external configuration management backed by a Git repository.
2. Service discovery: Integration with Eureka Services Discovery from Netflix OSS.
3. Circuit breakers: Fault-tolerant pattern to prevent cascading failures.
4. Intelligent routing: Spring Cloud Gateway is an intelligent and programmable router based on Spring Framework and Spring Boot.
5. Load balancing: Properly distributed network traffic to backend servers.

Features of spring cloud:

- Intelligent routing
- Services to services calls
- Distribution message
- Distribution configuration
- Load balancing

Advantages of Spring cloud:

- Standalone spring application in spring boot and distributed systems in spring cloud
- Embedded server in spring boot && service discovery, circuit breakers, routing, load balancing

Cloud Components:

- Configuration
- Service discovery
- Circuit breakers
- Routing and messaging
- API gateway
- Tracing
- CI pipeline and Testing

React

08 October 2024 11:16

1. Introduction to react: main purpose, framework, api
2. Why react, advantages of react
3. Features of react
4. Pros and cons of react
5. React js Vs Angular Js
6. React js Vs React Native
7. React js Vs Vue js
8. React js ...
9. React components (imp)
10. React states
11. React props
12. React props validation
13. React states vs React Props
14. React constructor
15. React components api
16. React components life cycle
17. React forms
18. Conditional rendering
19. React list
20. React keys
21. React fragments
22. React router
23. React css
24. React map
25. React table
26. React animation
27. React higher order component
28. React code splitting react context
29. React hooks
30. React flux
31. React redux
32. React axioms
33. React portal

Chapter 1: Introduction to JavaScript

- Data types
- Objects
- Operators
- Conditional Statements
- Control Statements
- Functions

Chapter 2: Strings

- Template Literals
- Escape Sequence Characters
- Arrays and its methods
- Loops

Chapter 3: JavaScript and the Web

- JavaScript usage
- Console Object methods
- Interaction of web page using JS
- Window Object BOM and DOM

Chapter 4: Document Object Model (DOM)

- Document Object Model
- Browser Object Model
- Auto Correction
- Children of an element
- DOM collection

Chapter 5: DOM Navigation and Tables

- Siblings and parents
- Element only navigation
- Table links
- DOM properties

Chapter 6: HTML and Attribute Manipulation

- Inner HTML/Outer HTML
- Attribute methods
- Insertion Methods
- Insert Adjacent HTML/Text/Element

Chapter 7: Class and Timers

- ClassName and ClassList
- Set Timeout and Set Interval

Chapter 8: Browser Events and Event Handling

- Browser Events
- Handling Events

- Listeners
- Event Object

Chapter 9: Callbacks, Promises, and Async/Await

- Callback function
- Handling errors
- Pyramid of Doom
- Promises
- Consumes—then and catch
- Attaching multiple handlers
- Promise API
- Async/await
- Error handling
- Try..catch
- The error object
- Throw custom Error
- Finally Clause

Chapter 10: Object-Oriented Programming

- Setting Prototype
- Classes and Objects
- Constructor
- Class Inheritance
- Extends
- Method Overriding
- Super keyword
- Overriding Constructor
- Static method
- Setters and getters
- Instance of operator

Chapter 11: Advanced JavaScript

- Destructuring
- Block Scopes
- Hoisting

Introduction

08 October 2024 11:58

- React is a declarative, efficient, flexible java script library for built reusable user interface components
- It is a open source
- It is responsible for only the view layer of application
- It was initially developed by meta
- Note: react is also called as framework which helps to build user interface
- Why react
- The main objective of react to develop a user interface improves speed of the application.
- React provides a feature called components, which can application split into several components
- Flow of data in uni directional

Advantages of react

- We can use DOM
- Cross platform development
- Search engine optimization

Features of react

- JSX
- Component based architecture
- One way data from parent to child
- Performance
- Dynamic routing: (static routing and dynamic routing)

Pros

git

02 August 2024 18:32

Version control
version control system
Centralized version control system
distributed version control system

```
C:\Users\murth>git config --global user.name "murthy30300"
```

```
C:\Users\murth>git config --global user.email "2200030300cseh@gmail.com"
```

```
C:\Users\murth>git config --global --list
user.email=2200030300cseh@gmail.com
user.name=murthy30300
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
gui.recentrepo=C:/Users/murth/OneDrive - K L
University/REACT/Workspace/cloud-workspace/My-web-development
```

1.Version Control

```
git init
git remote add origin <repo-link>
git remote remove origin
git remote -v
git config - -global user.name <name>
git config - -global user.email 2200031277@kluniversity.in
git add .
git commit -m "message"
git push - -set-upstream origin master
git branch
git branch <branch-name>
git checkout <branch-name>
git log
git checkout <commit-id>
```

There are some more important commands frequently used in git during work collaborations

Such as git merge and git rebase , but for now we will not be using them anyway we only will have 2 branches so no use of them too. But study about them when

you are free

git merge – used on master to feature

Combines changes from one branch into another, creating a new commit that preserves the history of both branches.

git rebase – used on feature to master

Integrates changes from one branch onto another by moving the entire branch to begin on the tip of the target branch, resulting in a linear commit history.

staging area

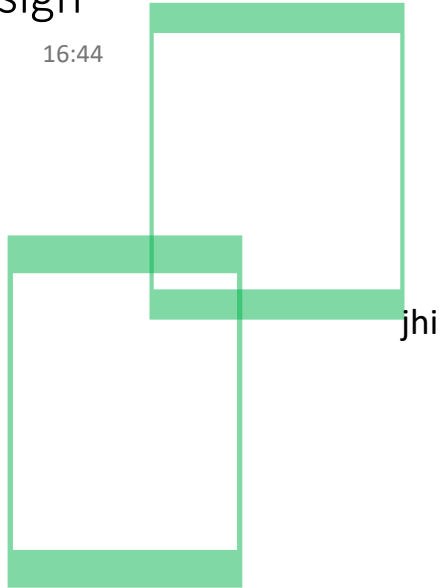
commit history

git init -b main

System design

29 September 2024

16:44



Core

02 November 2024

14:58

- Additional techniques to iterate in objects
- Arrays

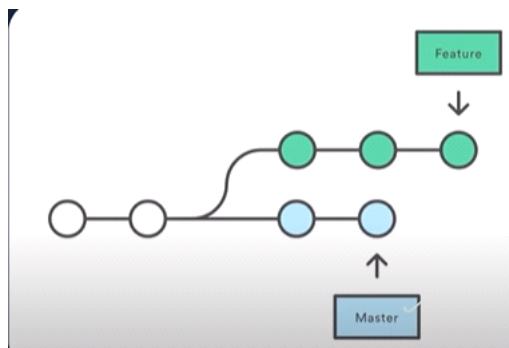
Table of contents

- Setup using config
- Init
- Stage and snapshot
- Branch and merge
- Inspect and compare
- Tracking path changes
- Ignoring patterns
- Share and update
- Rewrite history
- Temporary commits

Configuring git

- We can configure git into
 - o global and local
- ~ :- we are in root directory
- Git clone
- Git status
- Ls - a for hidden files
- Files in git
 - o Untracked file: new files that git does n't yet add
 - o Modified: changed
 - o Staged: file is yet to be commit
 - o Unmodified
- Add and commit
 - o Add
 - o Commit
- Git push
- To push code
 - o Git init
 - o Git remote add origin <> :- remote is git repo
 - o git remote -v : to check version
 - o git branch : current branch
 - If we working on a project which consists of many features, there are different teams who works on project like production, developer, testing teams. Each team can create a different branch and based on the correctness and completeness of code they can merge into main branch

- Git branch -M main: to rename branch
- git push -u origin main
 - -u: to set upstream, if we want to push the code into main branch always we will set first upstream then we can use "git push"
- Workflow of git:
 - Github repo-->clone--> changes--> add --> commit --> push
- Git branch
 - git branch -- to check current branch
 - git branch -M (to rename branch)
 - Git checkout <-branch name-> to navigate
 - Git checkout -b <-new branch name-> (to create new branch)
 - Git branch -d <-branch name-> to delete branch



- Merging code
 - 1st method:
 - git diff <branch name>: to compare commits, branches, files and more
 - git merge <- branch name-> (to merge 2 branches)
 - 2nd method :
 - Create pull request: pull requests tell others about changes you've pushed to a branch in a repository on GitHub
 - Pull command:
 - git pull origin main: used to fetch and download content from a remote repository and immediately update the local repo to match that content
- Merging conflicts:
 - An event that takes place when git is unable to automatically resolve differences in code between two commits
- Undoing changes:
 - The commits in the github store like a linked list data structure the most recent commit is termed as HEAD
 - Case-1: staged changes
 - Till commit-6 our code works fine every feature, we add another feature using git add . But later we realized there is an error, we didn't commit the file yet then we can use this command to revert the changes so that it can go back to commit-6
 - git reset <file name>: for single file

- ☐ git reset : for multiple files
 - Unfortunately we committed the changes as commit-7, to revert to commit six we use
 - ☐ git reset HEAD~1
 - After several commits we realized the bug in commit-7 we can revert to changes using commit hash values, those hash values we can get using git log command
 - ☐ git reset <- commit hash->
 - ☐ git reset --hard <- commit hash->
- Fork:
 - Fork is a new repository that shares code and visibility settings with the original "upstream" repository
 - Fork is rough copy